D0 Start here: foundations for finding errors

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track D: Finding errors in code 2
D0 Start here: foundations for finding errors 3
D0-L1 Bugs, errors and failures, the words and the method 3
D0-L2 Read code you did not write 7
Track D best-practice checklist 12
Appendix: solutions and answer keys 13

Track D: Finding errors in code

Find and fix errors fast, including errors in code that Claude and Copilot write.

D0 Start here: foundations for finding errors

Roadmap: Every month. The words and the method of debugging, and how to read code you did not write.

D0-L1 Bugs, errors and failures, the words and the method

Concept

This track is about finding what is wrong in code, quickly and for certain. Start with three kinds of problem, from cheapest to most expensive.

- A **syntax error** stops Python before anything runs. It points at the line.
- An **exception** stops the program while it runs, with a traceback that names the error type and the line, such as `KeyError: 'R2'`.
- A **wrong result** raises nothing. The code finishes and reports the wrong number. These **silent bugs** cost most, because nobody knows to look.

Use precise words. A **defect** (bug) is the mistake in the code. A **failure** is what someone sees: a crash, or a total that is 3% too high. The **symptom** is where it shows. The **root cause** is the first place it went wrong, often far from the symptom.

Work through every bug with the same method:

1. **Reproduce** it, then shrink the input to the smallest one that still fails. Three rows beat three million.
2. **Read the evidence**: the last line of the traceback, or the first value that differs from what you expected.
3. **Form one hypothesis** that would explain everything you see.
4. **Test it** with one small check: an `assert`, a print of a single value, or the debugger (lesson D2-L1).
5. **Write a test** that fails because of the bug, so it cannot return unnoticed.
6. **Fix the root cause**, not the symptom, and watch the test pass.
7. **Look for the same mistake elsewhere.** Copied code copies bugs.

Two habits undo the method. Changing several things at once means you never learn which change mattered. Catching every exception and returning a default, such as `except Exception: return 0.0`, turns a loud failure into a silent one.

Why it matters

A weight loader caught every exception and returned 0.0 so that "the pipeline never breaks". When a supplier changed its file encoding, every read failed, every weight became 0.0, and the gold tables showed zero respondents for three regions. The pipeline reported success for a week. Without the catch-all, the first run would have stopped with a clear `UnicodeDecodeError` naming the file.

Worked example: weak code

```
def mean_weight(path: str) -> float:
    try:
        values = [float(line) for line in open(path)]
        return sum(values) / len(values)
    except Exception:
        return 0.0
```

1. **Line 5.** Catches everything: a missing file, a bad number, an empty file, a typo in the code.
2. **Line 6.** A plausible default. The failure becomes a silent wrong result.

Worked example: strong code

```
from pathlib import Path


def mean_weight(path: Path) -> float:
    """Mean of the weights in a file with one number per line.

    Raises:
        ValueError: If the file holds no weights, or a line is not a number.
    """
    values = [float(line) for line in path.read_text().splitlines() if line.strip()]
    if not values:
        raise ValueError(f"no weights in {path}")
    return sum(values) / len(values)
```

1. **Line 10.** A bad line raises ValueError naming the text, and a missing file raises FileNotFoundError. Both are loud and specific.
2. **Line 12.** The empty case is handled on purpose, with a message that names the file.

Reproduce small, read the evidence, test one hypothesis at a time, pin the bug with a failing test, fix the cause, and never silence an error with a default.

Recap card

- **Three kinds of problem, cheapest first** Syntax error (nothing runs), exception (crash with a traceback), wrong result (silent bug).
- **Defect, failure, root cause** Defect: the mistake in code. Failure: what someone sees. Root cause: the first place it went wrong.
- **The debugging method** Reproduce small, read the evidence, one hypothesis, one check, a failing test, fix the cause, look for the same mistake elsewhere.
- **Why except Exception: return 0.0 is dangerous** It turns a loud, specific failure into a silent wrong result.

Practice

Exercise 1. Multiple choice (level 1)

A nightly job finishes without errors, but the gold table's weight total is 3% higher than the delivery's. What kind of problem is this?

- a) A syntax error
- b) An exception
- c) A wrong result, a silent bug
- d) Not a bug, because nothing failed

Why?

- r1) The code ran to the end and produced a wrong number, with no error to point at it.
- r2) Silent bugs are always caused by bad data.

Exercise 2. Put in order (level 2)

Put the debugging method in order.

- Write a test that fails because of the bug.
- Fix the root cause and watch the test pass.
- Test the hypothesis with one small check.
- Form one hypothesis that explains everything you see.
- Read the evidence: the last traceback line or the first wrong value.
- Reproduce the failure and shrink the input to the smallest one that still fails.

Exercise 3. Predict the output (level 2)

What does this print?

```python
weights = {"R1": 1.0}
try:
    print(weights["R2"])
except KeyError as err:
    print("missing key:", err)
```

Exercise 4. Write the function (level 3)

Step 1 of the method is to shrink a failing input. Write `minimise(rows: list[int], fails: Callable[[list[int]], bool]) -> list[int]`.

- Raise `ValueError` if `fails(rows)` is `False`: there is nothing to shrink.
- Otherwise, repeatedly try removing one element at a time, from the left. Whenever the shorter list still fails, keep it and start again.
- Stop when no single removal still fails, and return that list.

Keep the original order, and do not change `rows`.

Your code must pass these checks:

- For 'any negative value fails', [3, -1, 4, -5] shrinks to [-5] when you remove from the left and restart after each success.
- For 'sum above 10 fails', [4, 1, 9, 2, 6] shrinks to [9, 6]. Restart from the left after each removal.
- Raise ValueError when the full input does not fail.
- Work on a copy: leave the list passed in unchanged.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Shrink a failing input by removing one element at a time.
def minimise(
    rows: list[int],
    fails: Callable[[list[int]], bool],
) -> list[int]: ...
```

Starter code:

```python
def minimise(rows, fails):
    return rows
```

Exercise 5. Spot the bug (level 3)

A file of weights with one corrupted line makes this function report a total of 0.0, and the pipeline carries on. Tap the **two** lines that turn the failure into a silent wrong result, then fix the function so bad input raises.

```python
def load_total(lines: list[str]) -> float:
    """Total weight from the lines of a delivery file."""
    try:
        return sum(float(line) for line in lines)
```

```
    except Exception:
        return 0.0
```

Write down the line number of the bug. Then write the fix.

Sources

- A missing dict key raises KeyError whose message is the key's repr ('R2'); float("n/a") raises ValueError. (verified: Run on Python 3.11 and 3.13.)

D0-L2 Read code you did not write

Concept

Most code you debug was written by someone else: a colleague, your past self or an AI assistant. Read it in a fixed order, so you understand it before you trust it or change it.

1. **The contract.** Read the name, the signature and the docstring. What does it promise to take and return? `get_unprocessed_waves(...) -> list[str]` promises a list and no side effects.
2. **Inputs and outputs.** Which values come in, and what leaves: the return value, a written table, a printed line?
3. **Side effects.** Look for anything that changes the world: writing files or tables, network calls, `print`, and changes to arguments, such as `list.remove` or `df[col] = ...` on a frame passed in. A function named `get_` or `check_` should have none.
4. **Trace by hand.** Pick a tiny input, three values, and follow the code line by line. Write each variable's value in a small table after every step. It is slow for one minute and saves an hour.
5. **Edges.** Ask what happens with an empty input, a zero, a `None`, a duplicate key and one row. Most bugs live there.
6. **Assumptions.** Note what the code takes for granted: sorted input, unique IDs, a column that is never missing. Each one is a question for its author, or a check to add.

Only then run it, on a small input whose right answer you worked out by hand.

While you read, watch for code that changes a list while looping over it. `for w in weights: weights.remove(w)` skips the element after each removal, so some values survive.

If the code is hard to read, restate it: rename variables, add type hints, and split it into small functions. Rewriting it clearly, with the same behaviour, is often how you find the bug.

Why it matters

A helper called `clean_weights` was reused in four notebooks. It also sorted the caller's list in place, which nobody noticed from its name. One notebook relied on the original order to pair weights with respondent IDs, and every respondent after the first got someone else's weight. Reading the helper for side effects, step 3 above, finds the `sort()` in seconds.

Worked example: weak code

```
1  def f(a, b):
2      r = []
3      for i in range(len(a)):
4          if a[i] is not None and b[i] > 0:
5              r.append(a[i] / b[i])
6      return r
```

1. **Line 1.** No contract: the name, the parameters and the result tell a reader nothing.
2. **Line 3.** Assumes b is at least as long as a, and says nothing if it is longer.
3. **Line 4.** Silently drops pairs, so the result can be shorter than the inputs.

Worked example: strong code

```
1  def ratios(numerators: list[float | None], denominators: list[float]) ->
       list[float]:
2      """Ratio of each pair, skipping missing numerators and non-positive
       denominators.
3
4      Raises:
5          ValueError: If the lists differ in length.
6      """
7      return [
8          num / den
9          for num, den in zip(numerators, denominators, strict=True)
10         if num is not None and den > 0
11     ]
```

1. **Line 1.** The restated contract: names and types say what goes in and what comes out.
2. **Line 9.** strict=True raises on unequal lengths, an assumption the original never checked.
3. **Line 10.** The skipping rule, now visible and documented.

Read the contract, the inputs and outputs and the side effects first, trace a tiny input by hand, then probe the edges and list the assumptions.

Recap card

- **Order for reading unfamiliar code** Contract, inputs and outputs, side effects, a hand trace on a tiny input, edges, assumptions. Then run it on a known answer.
- **Side effects to look for** Writes to files or tables, network calls, prints, and changes to arguments, such as list.remove or df[col] = ... on a frame passed in.
- **Why not remove items while looping over a list** Each removal shifts the rest left, so the loop skips the next element.
- **How to trace by hand** Pick three values and write each variable's value in a table after every step.

Practice

Exercise 1. Multiple choice (level 1)

You are handed a 60-line function to review. What do you read first?

- a) Its name, signature and docstring: what it promises.
- b) The most complicated loop, because bugs hide there.
- c) Nothing: run it on production data and see what happens.
- d) The last line, because that is what it returns.

Why?

- r1) You cannot judge whether code is right until you know what it is meant to do.
- r2) The signature is the shortest part.

Exercise 2. Predict the output (level 2)

Trace this by hand. What does it print?

```
def running_max(values):
    best = None
    out = []
    for v in values:
        if best is None or v > best:
            best = v
        out.append(best)
    return out


```

```
print(running_max([3, 1, 4, 1, 5]))
```

Exercise 3. Fill the gap (level 2)

Trace this code by hand and fill in `total` after each step.

```
total = 0
for v in [2, 5, 1]:
    total = total * 2 + v
```

```
step 1: v = 2, total = _____(1)
step 2: v = 5, total = _____(2)
step 3: v = 1, total = _____(3)
```

Exercise 4. Refactor (level 3)

Restate `f` clearly, with the same behaviour, as `ratios(numerators, denominators)`:

- type hints and a docstring,
- `zip(..., strict=True)` instead of indexing, so lists of different lengths raise `ValueError`,
- a list comprehension that skips a missing numerator or a denominator that is not positive.

Your code must pass these checks:

- Skip a missing numerator and a non-positive denominator, and divide the rest: [0.5, 0.5].
- A numerator of 0.0 is a real value. Test for a missing one with `is None`.
- Empty inputs give an empty list.
- Lists of different lengths must raise ValueError. zip(..., strict=True) does that.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Ratio of each pair, skipping missing numerators and non-positive
    denominators.
def ratios(
    numerators: list[float | None],
    denominators: list[float],
) -> list[float]: ...
```

Starter code:

```
def f(a, b):
    r = []
    for i in range(len(a)):
        if a[i] is not None and b[i] > 0:
```

```
            r.append(a[i] / b[i])
    return r
```

Exercise 5. Spot the bug (level 3)

`drop_negative([-1, -2, 3])` returns `[-2, 3]`, and it changes the caller's list too. Tap the line with the bug, then fix it so it returns a new list and leaves the input alone.

```
def drop_negative(weights: list[float]) -> list[float]:
    """Return the weights without negative values."""
    for w in weights:
        if w < 0:
            weights.remove(w)
    return weights
```

Write down the line number of the bug. Then write the fix.

Sources

- Removing items from a list inside a for loop over it skips elements: drop_negative([-1, -2, 3]) returns [-2, 3]; zip(..., strict=True) raises ValueError on unequal lengths (Python 3.10 and later). (verified: Run on Python 3.11 and 3.13.)

Track D best-practice checklist

☐ Read a traceback from the bottom up. Find the first frame in your own code.

☐ Reproduce the error on the smallest data that shows it.

☐ Check row counts before and after every join.

☐ Test numbers with `is None`, never with truthiness, when zero is a valid value.

☐ Never use a mutable object as a default argument.

☐ Check every function and parameter an AI suggests against the official docs for your version.

☐ Run AI code on a small input with a known answer before you trust it.

☐ Write the failing test first. A test that only repeats the code proves nothing.

☐ Ask the model to list its assumptions. Check each one.

☐ State versions, constraints, expected outputs and tests in every prompt.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

D0-L1 Bugs, errors and failures, the words and the method

D0-L1 exercise 1

Hints. (1) Did anything stop the job? (2) Is the result right? (3) It ran and gave the wrong number.

Answer. c) A wrong result, a silent bug Because: r1) The code ran to the end and produced a wrong number, with no error to point at it.

Code that runs and reports the wrong number is a silent bug, the most expensive kind, because nothing tells you to look.

Common wrong answers.

- `a`: A syntax error stops the code before it runs. This job finished.
- `b`: An exception stops the run with a traceback. There was none.
- `d`: A wrong total is a failure, even when nothing crashed.
- `c|r2`: Right kind, wrong reason. Silent bugs come from code as often as from data.

D0-L1 exercise 2

Hints. (1) You cannot study what you cannot make happen. (2) Evidence, then one idea, then a check of that idea. (3) The test must fail before the fix, so it comes first.

Answer.

1. Reproduce the failure and shrink the input to the smallest one that still fails.
2. Read the evidence: the last traceback line or the first wrong value.
3. Form one hypothesis that explains everything you see.
4. Test the hypothesis with one small check.
5. Write a test that fails because of the bug.
6. Fix the root cause and watch the test pass.

Reproduce, read, hypothesise and check. Then write a failing test, and fix the cause until it passes.

Common wrong answers.

- Reproduce first. Evidence you cannot reproduce cannot confirm a fix.
- Test the hypothesis before you change code, or you may fix the wrong thing.
- Write the test first. A test written after the fix never proved it could fail.

D0-L1 exercise 3

Hints. (1) What happens when you index a dict with a key it does not have? (2) The except block catches it and prints the error. (3) The error shows the key as Python writes it.

Answer.

```
missing key: 'R2'
```

`weights["R2"]` raises `KeyError`, and the error prints as the key's repr, `'R2'`, which tells you exactly which key was missing.

Common wrong answers.

- `missing key: R2`: A KeyError's message is the key's repr, so a string key prints in quotes.
- `None`: Indexing a dict with a missing key raises KeyError. Only `.get` returns None.

D0-L1 exercise 4

Hints. (1) Check `fails(rows)` first, then work on `current = list(rows)`. (2) `candidate = current[:i] + current[i + 1:]` is the list without element i. (3) Loop while the last pass removed something. After each successful removal, break and start again from the left.

Answer.

```python
from collections.abc import Callable


def minimise(rows: list[int], fails: Callable[[list[int]], bool]) -> list[int]:
    """Shrink a failing input by removing one element at a time.

    Args:
        rows: An input that makes fails return True.
        fails: Returns True when the bug shows on the given input.

    Returns:
        A smaller list that still fails, where no single removal still fails.

```

```
    Raises:
        ValueError: If the full input does not fail.
    """
    if not fails(rows):
        raise ValueError("the input does not fail, so there is nothing to shrink")
    current = list(rows)
    shrunk = True
    while shrunk:
        shrunk = False
        for i in range(len(current)):
            candidate = current[:i] + current[i + 1 :]
            if fails(candidate):
                current = candidate
                shrunk = True
                break
    return current
```

Repeated single removals keep the failing behaviour while the input shrinks. A small failing input makes the evidence easy to read and the test easy to write.

D0-L1 exercise 5

Hints. (1) What happens when `float("n/a")` runs? (2) Where does that error go? (3) The catch-all and the default it returns.

Answer. The bug is on line 5, 6.

Fixed version:

```
def load_total(lines: list[str]) -> float:
    """Total weight from the lines of a delivery file."""
    return sum(float(line) for line in lines)
```

`except Exception` swallows the ValueError, and `return 0.0` replaces it with a believable number. Remove both, so a corrupted file stops the run with a message naming the bad text.

Common wrong answers.

- Line 4: Summing the parsed lines is correct. The problem is what happens when parsing fails.
- Line 6: Returning 0.0 is half of it. Which line catches the error in the first place?
- Line 5: Catching everything is half of it. Which line hides the failure behind a plausible number?

D0-L2 Read code you did not write

D0-L2 exercise 1

Hints. (1) What do you compare the code against? (2) What the function promises. (3) Name, signature, docstring.

Answer. a) Its name, signature and docstring: what it promises. Because: r1) You cannot judge whether code is right until you know what it is meant to do.

The contract defines what "correct" means. Every later step compares the code with it.

Common wrong answers.

- `b`: Without the contract, you cannot tell a bug from intended behaviour.
- `c`: Running unread code on production data can change or delete real data.
- `d`: The return line makes sense only once you know what the function promises.
- `a|r2`: Right first step, wrong reason. It comes first because it defines what right means.

D0-L2 exercise 2

Hints. (1) Keep a table with v, best and out after each step. (2) Step 1: v 3, best 3, out [3]. Step 2: v 1, best stays 3. (3) out grows by one value on every step.

Answer.

```
1 [3, 3, 4, 4, 5]
```

`best` changes only when a larger value arrives, and `out` records it on every step: 3, 3, 4, 4, 5.

Common wrong answers.

- `[3, 4, 5]`: `out.append(best)` runs on every step, not only when best changes, so the list has five values.
- `[3, 1, 4, 1, 5]`: Each step appends the best value so far, not the current value.
- `5`: The function returns the whole list `out`, not only the final maximum.

D0-L2 exercise 3

Hints. (1) Start from total = 0. (2) Step 1: 0 x 2 + 2. (3) Step 2: 2 x 2 + 5. Step 3: 9 x 2 + 1.

Answer.

```
step 1: v = 2, total = 2
step 2: v = 5, total = 9
step 3: v = 1, total = 19
```

Each step doubles the previous total and adds v: 2, then 9, then 19.

Common wrong answers.

- g1 = `2`; g2 = `7`; g3 = `8`: That is a plain running sum. Each step doubles the old total before adding v.
- g1 = `4`; g2 = `13`; g3 = `27`: The doubling applies to the old total, not to v: 0 x 2 + 2 is 2.

D0-L2 exercise 4

Hints. (1) `for num, den in zip(numerators, denominators, strict=True)` pairs the values. (2) Keep a pair when `num is not None and den > 0`. (3) Put it in one list comprehension and return it.

Answer.

```python
def ratios(numerators: list[float | None], denominators: list[float]) -> list[float]:
    """Ratio of each pair, skipping missing numerators and non-positive denominators.

    Args:
        numerators: Values to divide, None when missing.
        denominators: Values to divide by.

    Returns:
        One ratio per usable pair, in order.

    Raises:
        ValueError: If the lists differ in length.
    """
    return [
        num / den
        for num, den in zip(numerators, denominators, strict=True)
        if num is not None and den > 0
    ]
```

The restated function says what it does in its name, types and docstring, and `strict=True` turns a silent assumption into an error.

D0-L2 exercise 5

Hints. (1) Trace it: which index does the loop look at after removing -1? (2) Removing an element shifts the rest left, so the loop skips the next one. (3) Build a new list with a comprehension instead.

Answer. The bug is on line 5.

Fixed version:

```
1 def drop_negative(weights: list[float]) -> list[float]:
2     """Return the weights without negative values."""
3     return [w for w in weights if w >= 0]
```

`remove` shifts the remaining elements left while the loop moves on, so -2 is never checked. It also changes the caller's list, a side effect the name does not promise. A comprehension fixes both.

Common wrong answers.

- Line 3: Looping over the list is fine on its own. The trouble starts when the loop body changes the same list.
- Line 4: The condition is right: -1 and -2 are negative.